

WINDOW FUNCTIONS-SQL

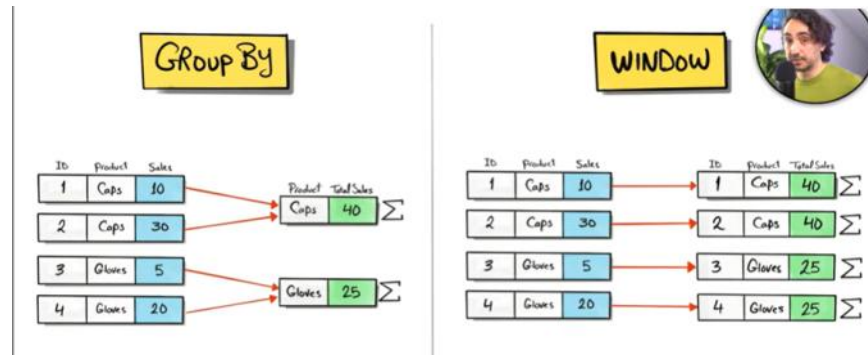
29 June 2026 15:26

WINDOW FUNCTIONS:-

Window Function performs calculations across a set of rows.

But does not collapse rows(unlike GROUP BY)

Perform calculations on a specific subset of data, without losing the level of details of rows.

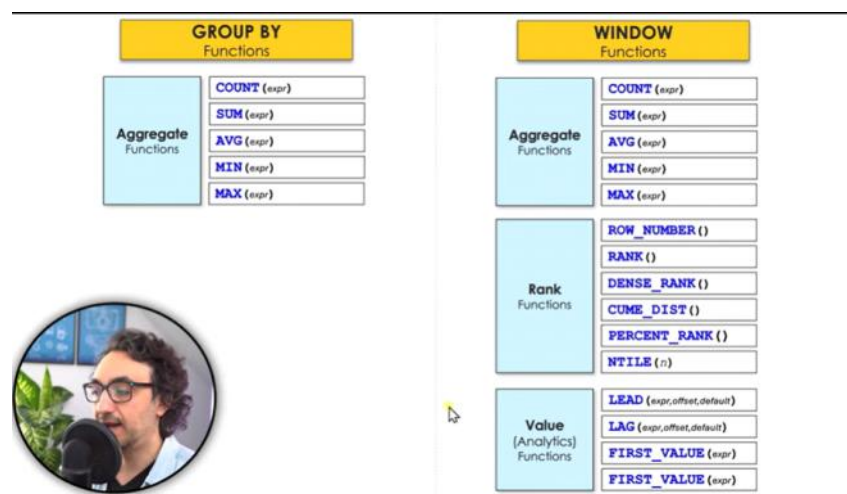


In case of Group By the granularity changes

In case of Window Function the granularity remains same

GROUP BY - Simple Aggregations

WINDOW- Aggregations + Keep details



Suppose we have a query like:-

Find the total sales for each product ,additionally provide details such order Id, order date.

- Here we need to use window functions:-

o SELECT

ORDERID,

ORDERDATE

PRODUCTID

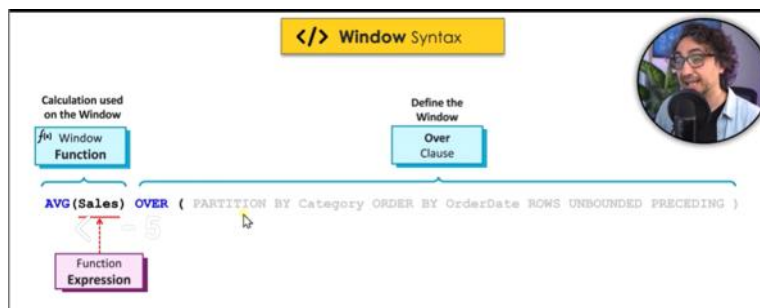
SUM(SALES) OVER(PARTITION BY PRODUCTID) TOTALSALESBYPRODUCTS

FROM SALES

- **OUTPUT:-**

	OrderID	OrderDate	ProductID	TotalSalesByProducts
1	1	2025-01-01	101	140
2	3	2025-01-10	101	140
3	8	2025-02-18	101	140
4	9	2025-03-10	101	140
5	10	2025-03-15	101	105
6	2	2025-01-05	102	105
7	7	2025-02-15	102	105
8	5	2025-02-01	104	75
9	6	2025-02-05	104	75
10	4	2025-01-20	105	60

But if we are doing the same thing through the group by function then we will be getting error or wrong data.



When using the **over clause**:- It tells that the function used is a window function. Used to define a window or subset of data.

PARTITION BY: Divides the rows into groups, based on the column/s.



Find the total sales across all orders additionally provide details such order id & order id and order date.

```
SELECT
OrderID,
OrderDate,
SUM(Sales) OVER() TotalSales
FROM Sales.Orders
```

OrderID	OrderDate	TotalSales
1	2025-01-01	380
2	2025-01-05	380
3	2025-01-10	380
4	2025-01-20	380
5	2025-02-01	380
6	2025-02-05	380
7	2025-02-15	380
8	2025-02-18	380
9	2025-03-10	380
10	2025-03-15	380

Find the total sales for each product additionally provide details such order id & order id and order date.

```
SELECT
OrderID,
OrderDate,
SUM(Sales) OVER(PARTITION BY ProductID) TotalSales
FROM Sales.Orders
```

	OrderID	OrderDate	TotalSales
1	1	2025-01-01	140
2	3	2025-01-10	140
3	8	2025-02-18	140
4	9	2025-03-10	140
5	10	2025-03-15	105
6	2	2025-01-05	105
7	7	2025-02-15	105
8	5	2025-02-01	75
9	6	2025-02-05	75
10	4	2025-01-20	60

Find the total sales for each combination of product and order status.

```
SELECT
SUM(SALES) OVER (PARTITION BY PRODUCTID,ORDERSTATUS) TotalSales
FROM Sales.Order
```

	OrderID	OrderDate	ProductID	OrderStatus	Sales	TotalSales	SalesByProducts	SalesByProductsAndStatus
1	1	2025-01-01	101	Delivered	10	380	140	30
2	3	2025-01-10	101	Delivered	20	380	140	30
3	8	2025-02-18	101	Shipped	90	380	140	110
4	9	2025-03-10	101	Shipped	20	380	140	110
5	7	2025-02-15	102	Delivered	30	380	105	30
6	10	2025-03-15	102	Shipped	60	380	105	75
7	2	2025-01-05	102	Shipped	15	380	105	75
8	5	2025-02-01	104	Delivered	25	380	75	75
9	6	2025-02-05	104	Delivered	50	380	75	75
10	4	2025-01-20	105	Shipped	60	380	60	60

Rank each order based on their sales from highest to lowest, additionally provide details such order id & order date

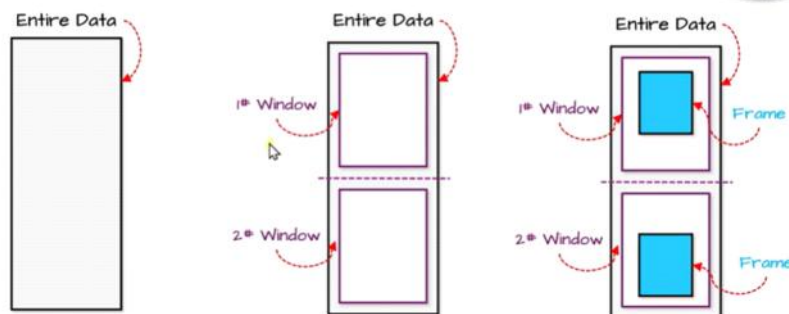
```
SELECT
OrderID,
OrderDate,
SALES,
RANK() OVER( ORDER BY SALESDesc) RankSales
FROM Sales.Orders
```

	OrderID	OrderDate	Sales	RankSales
1	8	2025-02-18	90	1
2	4	2025-01-20	60	2
3	10	2025-03-15	60	2
4	6	2025-02-05	50	4
5	7	2025-02-15	30	5
6	5	2025-02-01	25	6
7	9	2025-03-10	20	7
8	3	2025-01-10	20	7
9	2	2025-01-05	15	9
10	1	2025-01-01	10	10

Query executed successfully.

WINDOW FRAME

Defines a subset of rows within each window that is relevant for the calculation.



Frame Clause | Syntax

`AVG(Sales) OVER (PARTITION BY Category ORDER BY OrderDate`
`ROWS BETWEEN CURRENT ROW AND UNBOUNDED PRECEDING)`

Frame Types

- ROWS
- RANGE

Frame Boundary (Lower Value)

- CURRENT ROW
- N PRECEDING
- UNBOUNDED PRECEDING

Frame Boundary (Higher Value)

- CURRENT ROW
- N FOLLOWING
- UNBOUNDED FOLLOWING

Rules

- Frame Clause can only be used **together** with order by clause.
- Lower Value must be **BEFORE** the higher Value.

CURRENT ROW- It means that the starting/ending pointer is at the 1st row of that respective window.

N PRECEDING- It means include n rows above the current row

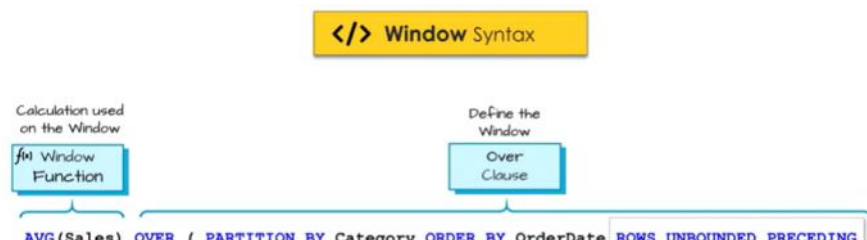
N FOLLOWING- It means include n rows below the current row

UNBOUNDED PRECEDING- It means include all the rows before the current row

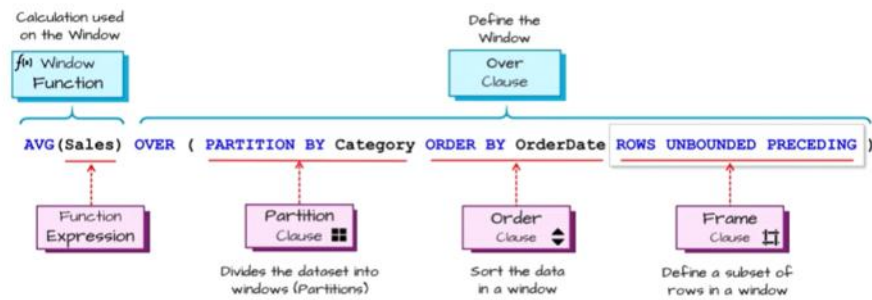
UNBOUNDED FOLLOWING- It means include all the rows after the current row

DEFAULT FRAME:- When you are not using window frame the default is :-

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW



</> Window Syntax



RULES OF USING WINDOW FUNCTIONS:-

- Window functions can be used only in **SELECT** and **ORDER BY** Clauses

```
SELECT
  OrderID,
  OrderDate,
  OrderStatus,
  Sales,
  SUM(Sales) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders
ORDER BY SUM(Sales) OVER (PARTITION BY OrderStatus) DESC
```

OrderID	OrderDate	OrderStatus	Sales	TotalSales	
1	8	2025-02-18	Shipped	90	245
2	9	2025-03-10	Shipped	20	245
3	10	2025-03-15	Shipped	60	245
4	4	2025-01-20	Shipped	60	245
5	2	2025-01-05	Shipped	15	245
6	1	2025-01-01	Delivered	10	135
7	3	2025-01-10	Delivered	20	135
8	5	2025-02-01	Delivered	25	135
9	6	2025-02-05	Delivered	50	135
10	7	2025-02-15	Delivered	30	135

```
SELECT
  OrderID,
  OrderDate,
  OrderStatus,
  Sales,
  SUM(Sales) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders
WHERE SUM(Sales) OVER (PARTITION BY OrderStatus) > 100
```

Msg 4108, Level 15, State 1, Line 8
Windowed functions can only appear in the SELECT or ORDER BY clauses.

Completion time: 2024-03-02T13:59:08.4640664+01:00

- Nesting window functions is not allowed

```
SELECT
  OrderID,
  OrderDate,
  OrderStatus,
  Sales,
  SUM(SUM(Sales) OVER (PARTITION BY OrderStatus)) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders
```

Msg 4109, Level 15, State 1, Line 6
Windowed functions cannot be used in the context of another windowed function or aggregate.

Completion time: 2024-03-02T14:01:09.1059043+01:00

- SQL execute WINDOW functions after WHERE Clause
- Window Function can be used together with GROUP BY in the same query, ONLY if the same columns are used.

SQL query to rank customers based on their total sales:

```

SELECT
  CustomerID,
  SUM(Sales) TotalSales,
  RANK() OVER(ORDER BY SUM(Sales) DESC) RankCustomers
FROM Sales.Orders
GROUP BY CustomerID

```

	CustomerID	TotalSales	RankCustomers
1	3	125	1
2	1	110	2
3	4	90	3
4	2	55	4

SQL WINDOW FUNCTIONS

Performs calculations on subset of data without losing details

Window v/s GroupBy

- Window is more powerful & dynamic than GroupBy
- Data Analysis → Advanced → Window
→ Simple → GroupBy
- Use GroupBy + window in same query, only if same columns used.

Components

Window Functions + Window Definition OVER

Rules

- Nesting is not allowed!
- Window can be used only in **SELECT** and **ORDER BY**
- SQL executes window AFTER **filtering** data using **WHERE**

Divide Data → Sort Data → Define Subset
PARTITION BY ORDER BY FRAME